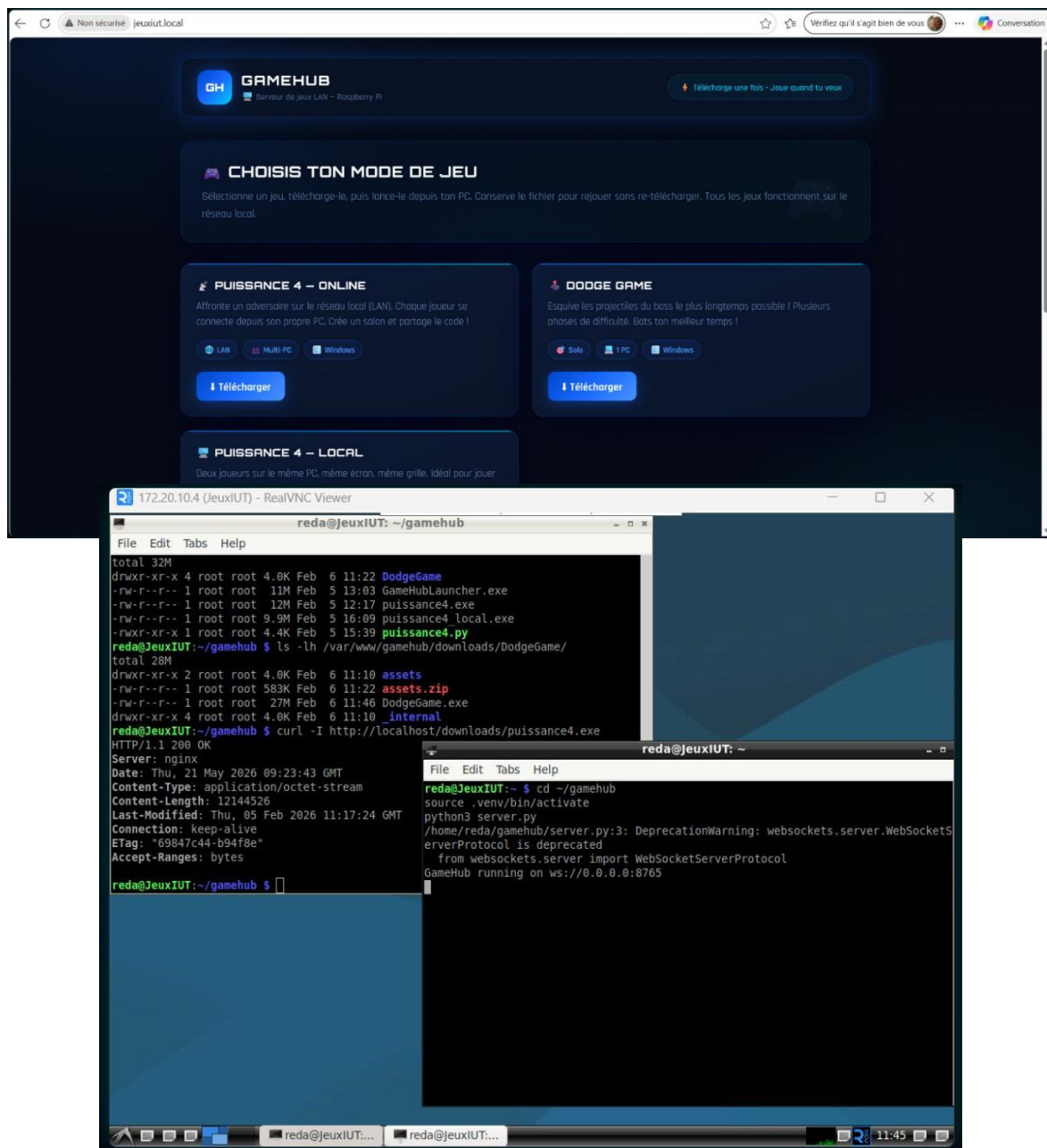




Projet tutoré :

GameHub

(Serveur de jeux LAN sur Raspberry Pi 3)



BUT GEII-3 All

Sommaire

1. Le point de départ : le sujet et les contraintes	3
2. Ce que nous avons mis en place	5
3. Les jeux disponibles sur la plateforme	7
4. Les obstacles rencontrés et comment nous les avons surmontés	9
5. Bilan	11

1. Le point de départ : le sujet et les contraintes

Dans le cadre de notre projet tutoré de deuxième année de BUT GEII, notre enseignant nous a confié un sujet volontairement ouvert : créer un serveur permettant à d'autres étudiants d'accéder à des jeux développés par leurs camarades et d'y jouer. Aucune technologie n'était imposée, c'était à nous de décider comment faire.

Pour donner une image concrète : l'idée était de créer quelque chose de similaire à Steam, la célèbre plateforme de jeux vidéo, mais en version locale, accessible uniquement depuis le réseau de l'IUT, et hébergé sur un Raspberry Pi, un petit ordinateur de la taille d'une carte de crédit, peu coûteux mais parfaitement adapté à l'hébergement de services réseau.

Le matériel mis à notre disposition

- **Raspberry Pi 3**, un petit ordinateur sous Linux utilisé comme serveur central
- **Routeur TP-Link**, utilisé pour connecter tous les PC de la salle sur le même réseau local
- Aucun budget, aucun logiciel imposé. Tous les choix techniques étaient libres

Ce que nous devons résoudre

- Comment les étudiants vont-ils accéder aux jeux ? Via un site web ? Une application ?
- Comment faire jouer deux personnes ensemble depuis deux PC différents, en temps réel ?
- Les jeux des camarades sont des scripts Python. Comment les rendre utilisables sur Windows sans installation ?
- Comment faire fonctionner le serveur automatiquement, sans intervention manuelle chaque matin ?

Ainsi, nous avons décidé d'héberger un site web via **Nginx** sur le Raspberry Pi, permettant de télécharger les jeux compilés en .exe Windows. Pour le multijoueur, nous avons développé un serveur **WebSocket** en Python, un protocole de communication permettant des échanges en temps réel entre plusieurs PC.

2. Ce que nous avons mis en place

Un site web hébergé sur le Raspberry Pi

Nginx est un logiciel de serveur web : il permet à un ordinateur de répondre aux requêtes d'un navigateur, comme le ferait le serveur d'un vrai site internet. Nous l'avons installé sur le Raspberry Pi pour qu'il serve la page GameHub et les fichiers de jeux à télécharger.

Pour éviter que les étudiants aient à connaître l'adresse IP du Pi, qui peut changer selon le réseau, nous avons installé **Avahi**, un service qui implémente le protocole **mDNS**. Concrètement, cela permet d'accéder au Pi via l'adresse **http://jeuxiut.local**, comme un vrai nom de domaine, mais qui fonctionne uniquement sur le réseau local.

Un système de téléchargement de jeux

PyInstaller est un outil qui transforme un script Python en un fichier **.exe Windows** autonome, contenant Python et toutes les dépendances nécessaires. Les étudiants téléchargent ce fichier depuis le site et le lancent d'un double-clic, sans rien installer.

Un serveur de jeu multijoueur

Pour permettre à deux joueurs sur deux PC différents de jouer ensemble en temps réel, nous avons développé un serveur **WebSocket** en Python. Le WebSocket est un protocole de communication réseau qui maintient une connexion ouverte en permanence entre le client et le serveur. Contrairement à une page web classique où le navigateur envoie une requête et attend une réponse, ici les deux parties peuvent s'envoyer des messages à tout moment, instantanément.

Le serveur est développé avec la bibliothèque **asyncio**, qui permet de gérer plusieurs connexions simultanées sans bloquer le programme. Chaque joueur connecté est géré en parallèle sans qu'ils se gênent mutuellement.

Un démarrage automatique

Nous avons configuré un service **systemd** sur le Raspberry Pi. Systemd est le gestionnaire de services de Linux, qui permet de lancer des programmes automatiquement au démarrage du système. Notre service relance également le serveur WebSocket automatiquement en cas d'erreur (Restart=always).

Tableau récapitulatif de l'architecture

Fichier	Rôle	Technologie
server.py	Serveur WebSocket, gestion des connexions, rooms, parties	Python / asyncio / websockets
puissance4_online.py	Client multijoueur, interface graphique et connexion au serveur	Python / Tkinter / asyncio
puissance4_core.py	Logique pure du jeu (plateau, coups, victoire)	Python
index.html	Interface web GameHub, liste des jeux et téléchargements	HTML / CSS
gamehub.service	Démarrage automatique du serveur au boot du Raspberry Pi	systemd (Linux)

3. Les jeux disponibles sur la plateforme

Puissance 4 en réseau (développé par nous)

C'est la réalisation technique centrale du projet. Nous avons développé entièrement un jeu de Puissance 4 permettant à deux étudiants de s'affronter depuis deux PC différents, en temps réel via le réseau de l'IUT.

Concrètement : le premier joueur lance le jeu, entre son pseudo, et crée un salon. Le serveur génère un code à 4 caractères. Il communique ce code à son adversaire, qui le saisit dans son propre client pour rejoindre la partie. A chaque coup, le client envoie un message JSON au serveur WebSocket, qui vérifie la validité du coup, met à jour l'état de la partie et le rediffuse aux deux joueurs.

Ce que voit le joueur	Ce qui se passe côté serveur WebSocket
Il entre son pseudo	Authentification, vérification du mot de passe
Il crée un salon et reçoit un code	Génération d'un identifiant de room unique (4 caractères)
Son adversaire entre le même code	Assignation des slots joueur 1 et joueur 2
Il joue en cliquant sur une colonne	Vérification du coup, mise à jour du plateau, broadcast JSON
Il voit le coup de l'adversaire	Le serveur a retransmis l'état en temps réel au second client

Puissance 4 en local (développé par nous)

Version locale du même jeu, développée avec **Tkinter**, la bibliothèque d'interface graphique intégrée à Python. Deux joueurs jouent à tour de rôle sur le même PC, sans connexion réseau. La logique de jeu est partagée avec la version multijoueur via un module commun.

Dodge Game (jeu des camarades, corrigé et amélioré)

Le Dodge Game est un jeu d'esquive développé par deux autres étudiants de la promotion, Léandro et Gabriel, avec **Pygame**, une bibliothèque Python pour créer des jeux 2D. Le joueur pilote un vaisseau et doit éviter les projectiles d'un ennemi dont la difficulté augmente progressivement.

Le jeu ne fonctionnait pas sur d'autres machines que celle du développeur original. Voici les corrections et améliorations que nous avons apportées :

- **Correction des chemins d'assets** : les images et sons étaient chargés via des chemins absolus propres à la machine d'origine. Correction avec `resource_path()`, qui localise les ressources de façon relative à l'exécutable.
- **Ajout d'un écran de démarrage** : affichage des contrôles (Z/Q/S/D, Shift pour le sprint, Ctrl pour la marche lente) avant le début de la partie
- **Correction de la vitesse** : le déplacement était trop lent car `clock.tick(60)` manquait, ce qui laissait le jeu tourner sans limite de fréquence
- **Ajout d'un écran de fin** : avec le temps survécu, le meilleur temps de la session, et des boutons Rejouer / Quitter

4. Les obstacles rencontrés et comment nous les avons surmontés

Le jeu multijoueur ne démarrait jamais

Lors des premiers tests, les joueurs pouvaient créer ou rejoindre un salon, mais la partie ne démarrait jamais. En inspectant le code, nous avons découvert que le fichier .exe mis en téléchargement était la version locale du Puissance 4, sans aucune connexion WebSocket. Le client ne savait pas du tout comment communiquer avec le serveur.

Nous avons développé un nouveau client intégrant la connexion au serveur **WebSocket** directement dans l'interface graphique (**Tkinter**). La difficulté principale était que ces deux parties du programme voulaient chacune prendre le contrôle en même temps : l'interface attend les clics du joueur, tandis que la connexion réseau attend les messages du serveur. Nous avons résolu ce problème en les faisant fonctionner de façon indépendante en arrière-plan grâce à **asyncio**, sans que l'une bloque l'autre.

Le Dodge Game se fermait instantanément au lancement

En lançant l'exécutible depuis l'invite de commandes **CMD** Windows pour lire le message d'erreur, nous avons vu :

```
FileNotFoundError: 'e:/Gabriel/fond5.png'
```

Les images et sons étaient chargés via des chemins absolus propres à la machine du développeur. La correction a consisté à utiliser **resource_path()**, une fonction qui détecte si l'application tourne en mode script Python ou en mode exe compilé via **sys._MEIPASS** de PyInstaller, et construit le chemin correct dans chaque cas.

L'adresse IP du serveur changeait selon le réseau WiFi

L'adresse IP du Raspberry Pi était codée en dur dans le client : `SERVER_IP = "172.20.10.4"` (partage de connexion avec le téléphone iPhone de Reda). Dès que le Pi changeait de réseau, entre le partage de connexion téléphone et le routeur TP-Link, l'IP changeait et les clients ne pouvaient plus se connecter.

Nous avons installé **Avahi** sur le Pi, une implémentation du protocole **mDNS**, et remplacé l'IP par le nom d'hôte **jeuxiut.local** dans le code client. Ce nom est automatiquement résolu par le réseau local, quel que soit le réseau WiFi utilisé.

La fenêtre du jeu disparaissait à la saisie du pseudo

A l'ouverture du jeu, une petite boîte de dialogue s'affiche pour demander le pseudo du joueur. A ce moment-là, la fenêtre principale du jeu se réduisait automatiquement dans la barre des tâches. Les joueurs pensaient que le programme avait planté et le refermaient.

La correction a été simple : nous avons demandé à **Tkinter** de ramener automatiquement la fenêtre du jeu au premier plan dès que le joueur a validé son pseudo, comme quand on passe d'une fenêtre à une autre sur son bureau.

5. Bilan

Objectif	Résultat
Site accessible sans connaître l'IP (Nginx + Avahi/mDNS)	Accessible via http://jeuxiut.local sur tout réseau LAN
Jeux téléchargeables en .exe (PyInstaller)	3 jeux disponibles, téléchargement direct depuis le navigateur
Multijoueur temps réel (WebSocket + asyncio)	Puissance 4 Online fonctionnel avec rooms et synchronisation
Démarrage automatique au boot (systemd)	Service actif au branchement du Raspberry Pi
Intégration et correction du Dodge Game (Pygame)	Corrigé, amélioré et déployé sur la plateforme
Fonctionner sur plusieurs réseaux WiFi (mDNS)	jeuxiut.local fonctionne quel que soit le réseau

Dans ce projet, mettre en place une infrastructure réseau complète à partir d'un Raspberry Pi et d'un routeur, en choisissant librement les technologies, est très différent d'un exercice guidé en cours. Nous avons dû justifier chaque choix technique, faire face à des problèmes que l'on ne peut pas anticiper (IP dynamique, compatibilité Windows/Linux, communication réseau dans une interface graphique) et livrer une plateforme réellement utilisée par nos camarades.

La compétence principale que nous retenons est la capacité à décomposer un système complexe en briques indépendantes, à comprendre comment elles communiquent entre elles, et à déboguer méthodiquement en lisant les messages d'erreur, en isolant les problèmes, en testant étape par étape.